VEDAS COLLEGE

AFFILIATED TO TRIBHUVAN UNIVERSITY

JAWALAKHEL, LALITPUR

A

Lab Report

On

Multimedia Computing (CSC330)

Submitted By:

Name: Prajwal Chaulagain

Symbol No: 80010773

Semester: Fifth

Faculty: Science and Technology

Level: Bachelor

Program: Computer Science and Information Technology.

Submitted To:

Instructor: Indra Chaudhary

2026

BSC CSIT 5TH SEMESTER | MULTIMEDIA COMPUTING (CSC330)

Laboratory Assignment Log Sheet

Name: Prajwal Chaulagain **Roll No:** 28

Task No.	Lab Title	Signature
Sound and Audio		
1.	Sound Digitization & Visualization	
2.	Audio Modification Effects	
3.	Audio Editing & Synthesis	
Images and Graphics		
4.	Image Representation & Abstraction	
5.	Histogram Analysis	
6.	Image Dithering	
7.	Image Morphing	
Video and Animation		
8.	Video Editing & Signal Representation	
9.	Mathematical Animation	
10.	Timeline & Tweening Animation	
11.	GIF Creation	
Data Compression		
12.	Entropy Coding (RLE)	
13.	Entropy Coding (Huffman)	
14.	Lossy Compression (JPEG)	
Web Integration		
15.	Web-Based Multimedia Elements	

Task 1: Simulate Sound Digitization & Visualization.

Objective: To understand the fundamentals of digital audio by loading an existing audio file, displaying its waveform, and analyzing sampling and quantization.

Tasks: Write a Python program to simulate sound digitization. The program should:

• Accept an uploaded digital audio file (e.g., .wav) into the environment.

• Display the audio continuous waveform visually.

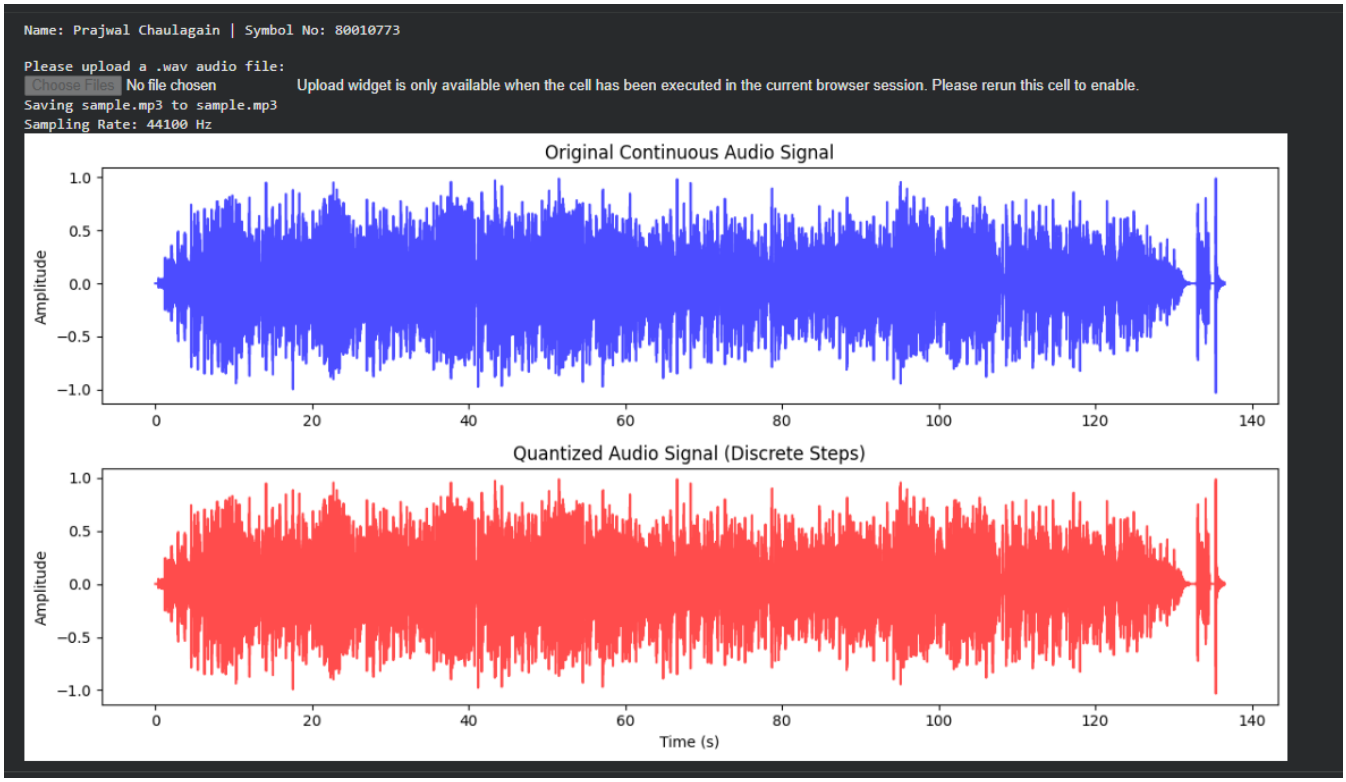
• Simulate and demonstrate the sampling and quantization processes (e.g., by reducing the bit depth and plotting the discrete steps).

Code:

```
# --- Package Installation ---
!pip install librosa matplotlib numpy
import librosa
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Mandatory output
print("Name: Prajwal Chaulagain | Symbol No: 80010773")
print("\nPlease upload a .wav audio file:")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]

# 1. Load the audio file (Sampling)
audio_data, fs = librosa.load(file_name, sr=None)
print(f"Sampling Rate: {fs} Hz")

# 2. Simulate Quantization (reducing to 8-bit discrete steps)
quantized_data = np.round(audio_data * 127) / 127.0

# Time axis for plotting
time = np.linspace(0, len(audio_data) / fs, len(audio_data))

# 3. Visualization
plt.figure(figsize=(12, 6))
```

```
plt.subplot(2, 1, 1)
plt.plot(time, audio_data, color='blue', alpha=0.7)
plt.title("Original Continuous Audio Signal")
plt.ylabel("Amplitude")
plt.subplot(2, 1, 2)
plt.step(time, quantized_data, color='red', alpha=0.7)
plt.title("Quantized Audio Signal (Discrete Steps)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()
```

Output:



Task 2: Simulate Audio Modification Effects.

Objective: To alter sound properties and understand digital audio manipulation effects.

Tasks: Write a Python program to apply audio modification effects. The program should:

• Accept an audio file as input.

• Apply amplifying, fade-in, and fade-out effects.

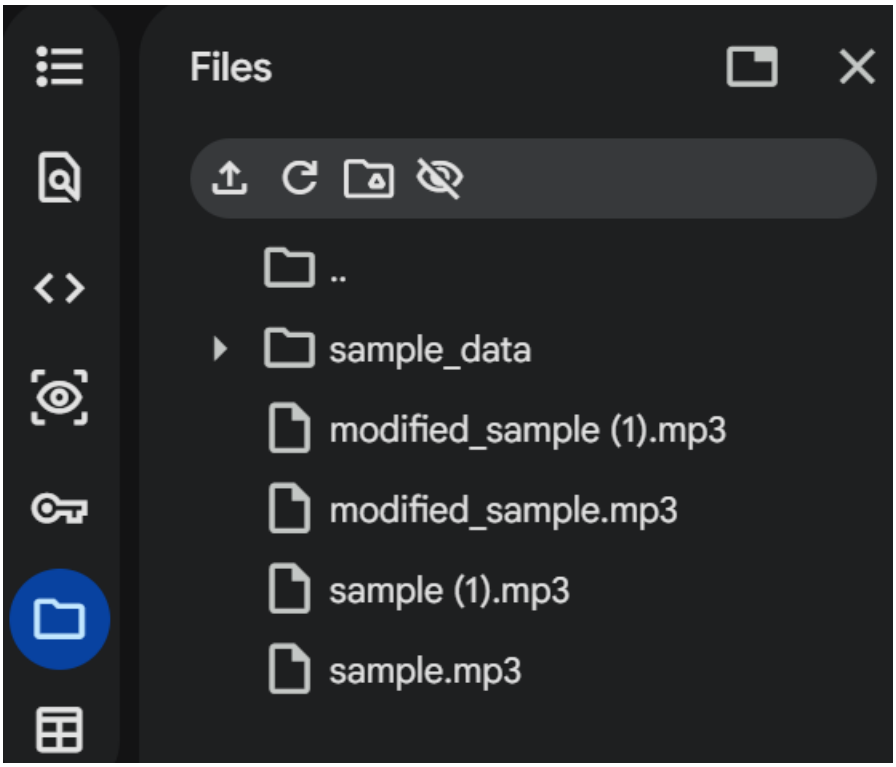
• Alter the pitch and speed of the audio, saving the modified file.

Code:

```
# --- Package Installation ---
!pip install pydub
from pydub import AudioSegment
from google.colab import files
# Mandatory output
print("Name: Prajwal Chaulagain\n")
print("Symbol No: 80010773\n")
print("Please upload an audio file (.wav or .mp3):")
# Upload the file
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
# Load the audio file
audio = AudioSegment.from_file(file_name)
# 1. Amplifying (Increase volume by 10dB)
amplified_audio = audio + 10
# 2. Fade-in & Fade-out (2 seconds each)
faded_audio = amplified_audio.fade_in(2000).fade_out(2000)
# 3. Pitch and Speed change (Speeding up by 1.5x)
# This will change both pitch and speed together.
modified_audio = faded_audio.speedup(playback_speed=1.5)
# Export the modified file
output_name = "modified_" + file_name
file_extension = output_name.split('.')[-1]
# If the original file is mp3, save the output in the same format
modified_audio.export(output_name, format=file_extension)
print (f"Audio effects applied successfully. Saved as '{output_name}'.")
```

Output:

```
Name: Prajwal Chaulagain

Symbol No: 80010773

Please upload an audio file (.wav or .mp3):
Choose Files sample.mp3
sample.mp3(audio/mpeg) - 1608431 bytes, last modified: 3/22/2026 - 100% done
Saving sample.mp3 to sample (1).mp3
Audio effects applied successfully. Saved as 'modified_sample (1).mp3'.
```



Task 3: Simulate Audio Editing & Synthesis.

Objective: To perform structural changes on audio files and implement Text-to-Speech synthesis.

Tasks: Write a Python program to edit audio and synthesize speech. The program should:

• Trim, cut, merge, and mix multiple audio tracks together.

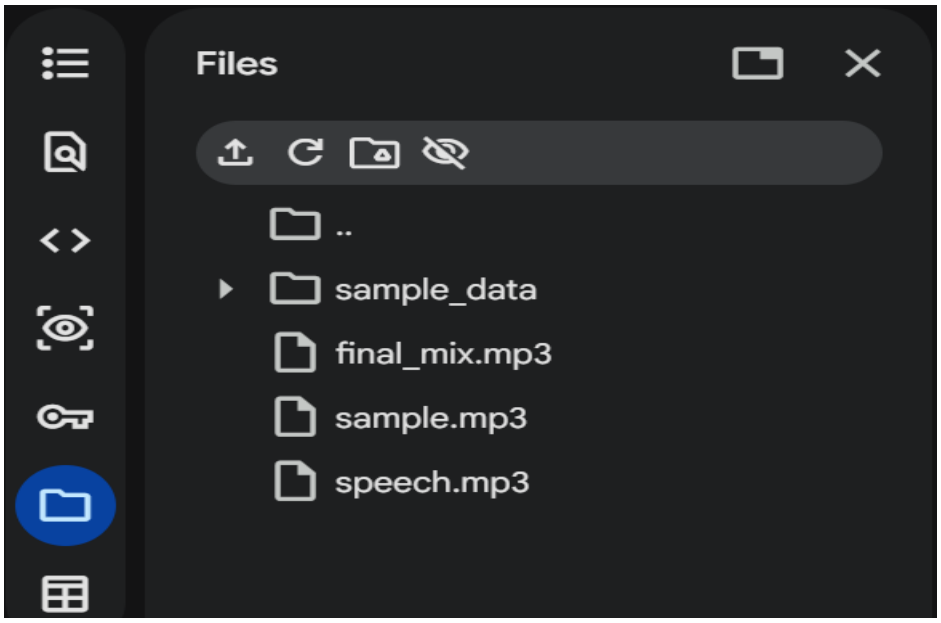
• Accept text input and synthesize it into spoken audio using Text-to-Speech (TTS).

Code:

```
# --- Package Installation ---
!pip install pydub gTTS
from pydub import AudioSegment
from gtts import gTTS
from google.colab import files
import os
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
# 1. Text-to-Speech Synthesis
text = "This is a synthesized voice for multimedia editing."
tts = gTTS(text=text, lang='en')
tts.save("speech.mp3")
print("TTS Synthesis complete. Saved as 'speech.mp3'.")
print("\nPlease upload a background audio file to mix:")
uploaded = files.upload()
bg_file = list(uploaded.keys())[0]
# 2. Audio Editing
speech = AudioSegment.from_file("speech.mp3")
background = AudioSegment.from_file(bg_file)
# Trim/Cut (Take the first 5 seconds of the background)
trimmed_bg = background[:5000]
# Mix (Overlay speech onto the trimmed background)
mixed_track = trimmed_bg.overlay(speech)

# Merge (Append the speech track again at the end)
final_track = mixed_track + speech

# Export
final_track.export("final_mix.mp3", format="mp3")
print("Audio editing complete. Saved as 'final_mix.mp3'.")
```

Output:

```
Name: Prajwal Chaulagain

Symbol No: 80010773

TTS Synthesis complete. Saved as 'speech.mp3'.

Please upload a background audio file to mix:
Choose Files sample.mp3
sample.mp3(audio/mpeg) - 1608431 bytes, last modified: 3/22/2026 - 100% done
Saving sample.mp3 to sample.mp3
Audio editing complete. Saved as 'final_mix.mp3'.
```



Task 4: Simulate Image Representation & Abstraction.

Objective: To process digital images using OpenCV and understand digital pixel representation without relying on live hardware.

Tasks: Write a Python program to process digital images. The program should:

• Accept and load an uploaded digital image file using OpenCV abstraction.

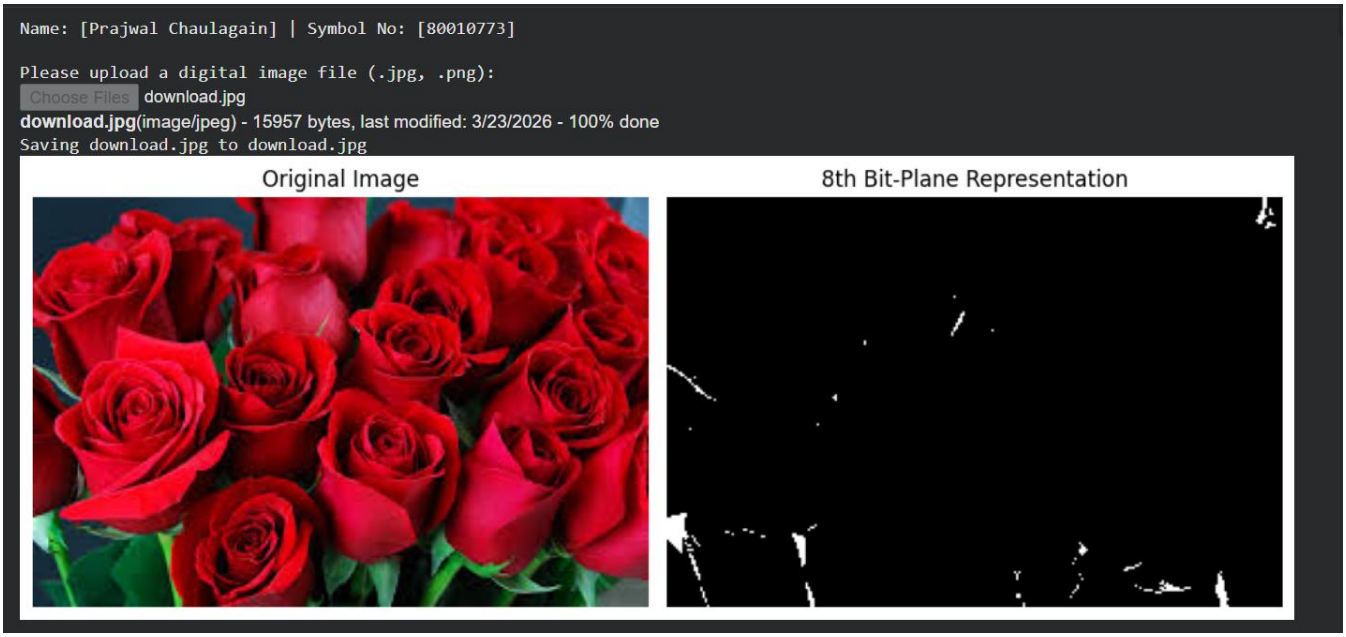
• Display the original loaded image on the screen.

• Process the image to demonstrate digital pixel representation (e.g., extracting and displaying the 8th bit-plane or splitting color channels).

Code:

```
# --- Package Installation ---
!pip install opencv-python matplotlib numpy
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
print("Please upload a digital image file (.jpg, .png):")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
# Load the image using OpenCV
image = cv2.imread(file_name)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Process digital pixel representation (Extract 8th Bit-Plane)
# 128 in binary is 10000000, isolating the Most Significant Bit
bit_plane_8 = cv2.bitwise_and(gray_image, 128)
# Display results
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(bit_plane_8, cmap='gray')
plt.title("8th Bit-Plane Representation")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:



Task 5: Simulate Histogram Analysis.

Objective: To calculate and plot pixel intensity distributions of digital images.

Tasks: Write a Python program to perform image histogram analysis. The program should:

• Accept a digital image as input.

• Calculate the pixel intensity distributions.

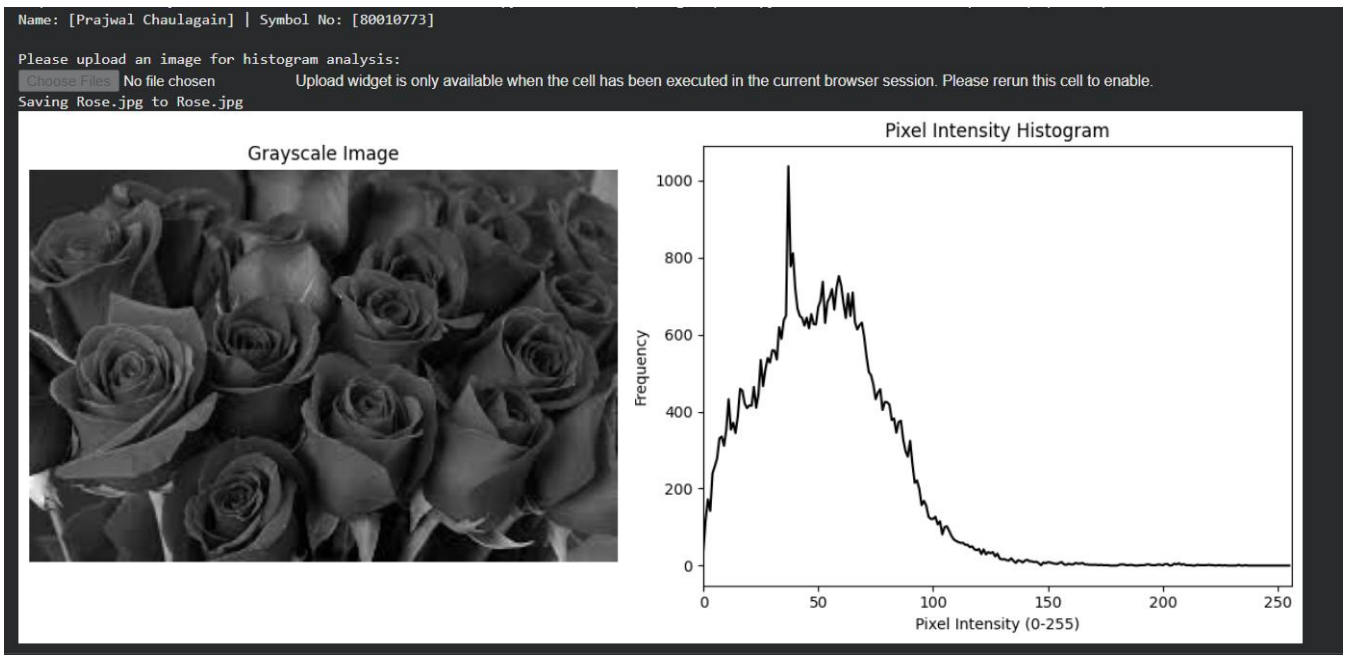
• Plot and display the resulting histogram alongside the image.

Code:

```
# --- Package Installation ---
!pip install opencv-python matplotlib numpy
import cv2
import matplotlib.pyplot as plt
from google.colab import files
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
print("Please upload an image for histogram analysis:")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
```

```
# Read image and convert to grayscale
image = cv2.imread(file_name)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Calculate Histogram
hist = cv2.calcHist([gray_image], [0], None, [256], [0, 256])
# Display image and histogram
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(gray_image, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.plot(hist, color='black')
plt.title("Pixel Intensity Histogram")
plt.xlabel("Pixel Intensity (0-255)")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.tight_layout()
plt.show()
```

Output:



Task 6: Simulate Image Dithering.

Objective: To implement 2×2 Cluster Dot Halftoning mathematically.

Tasks: Write a Python program to simulate image dithering. The program should:

• Accept a grayscale digital image as input.

• Apply a 2×2 Cluster Dot Halftoning algorithm to the image pixels.

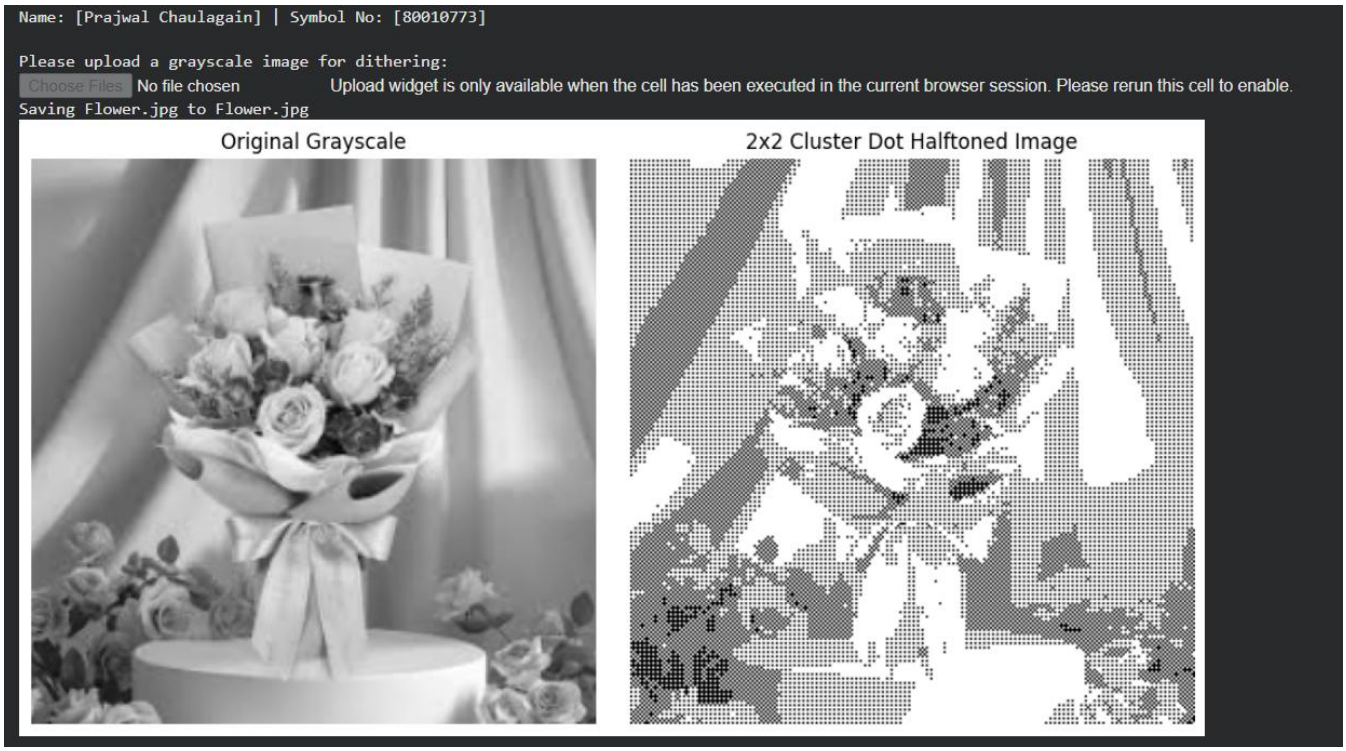
• Display the resulting dithered/halftoned image.

Code:

```
# --- Package Installation ---
!pip install opencv-python matplotlib numpy
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
print("Please upload a grayscale image for dithering:")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
# Read and resize for demonstration speed
image = cv2.imread(file_name, cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (256, 256))
# 2x2 Bayer Matrix for Cluster Dot Halftoning
bayer_matrix = np.array([[0, 128],
[192, 64]])
# Tile the matrix to match the image size
tiled_matrix = np.tile(bayer_matrix, (image.shape[0] // 2, image.shape[1] // 2))
# Apply the mathematical halftoning threshold
dithered_image = np.where(image > tiled_matrix, 255, 0)
# Display
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Grayscale")
```

```
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(dithered_image, cmap='gray')
plt.title("2x2 Cluster Dot Halftoned Image")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:



Task 7: Simulate Image Morphing.

Objective: To write and apply an algorithm that morphs one image into another.

Tasks: Write a Python program to morph images. The program should:

• Accept a source image and a target image.

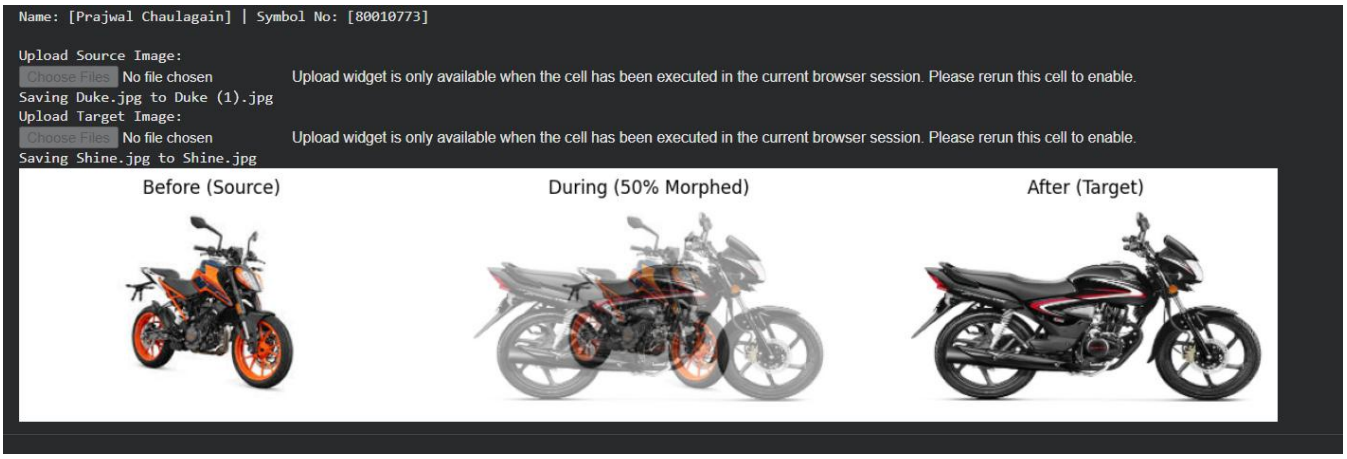
• Implement an algorithm to morph the source image into the target image.

• Provide and display before, during, and after examples of the morphing process.

Code:

```
# --- Package Installation ---
!pip install opencv-python matplotlib numpy
import cv2
import matplotlib.pyplot as plt
from google.colab import files
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
print("Upload Source Image:")
src_up = files.upload()
src_file = list(src_up.keys())[0]
print("Upload Target Image:")
tgt_up = files.upload()
tgt_file = list(tgt_up.keys())[0]
# Load images
img1 = cv2.cvtColor(cv2.imread(src_file), cv2.COLOR_BGR2RGB)
img2 = cv2.cvtColor(cv2.imread(tgt_file), cv2.COLOR_BGR2RGB)
# Resize to ensure exact dimensions for mathematical blending
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
# Implement linear morphing algorithm (50% blend)
alpha = 0.5
morphed_img = cv2.addWeighted(img1, 1 - alpha, img2, alpha, 0.0)
# Display Before, During, and After
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img1); axes[0].set_title("Before (Source)")
axes[0].axis('off')
axes[1].imshow(morphed_img); axes[1].set_title("During (50% Morphed)")
axes[1].axis('off')
axes[2].imshow(img2); axes[2].set_title("After (Target)")
axes[2].axis('off')
plt.show()
```

Output:



Task 8: Simulate Video Editing & Signal Representation.

Objective: To analyze video signal properties and perform basic video editing operations.

Tasks: Write a Python program to edit video files. The program should:

• Calculate and display the frame rate and total frame count of a video.

• Trim specific unwanted sections from the video.

• Overlay custom text onto the video frames and save the output.

Code:

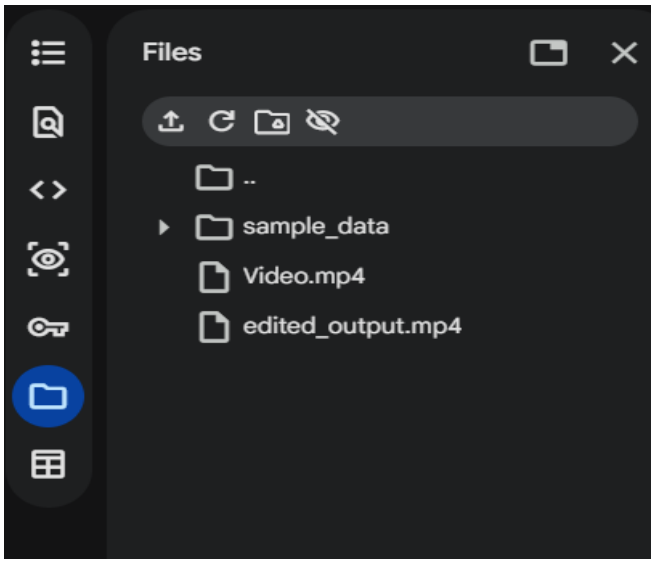
```
import os
# Install imagemagick if not already installed
# This is a common dependency for moviepy for image manipulation tasks.
# We'll use a check to avoid reinstalling on every run.
if not os.path.exists('/usr/bin/convert'):
    print("Installing ImageMagick...")
    !apt-get update -qq > /dev/null && apt-get install -y imagemagick
    print("ImageMagick installed.")


# Set the IMAGEMAGICK_BINARY environment variable.
# MoviePy uses this to locate the ImageMagick 'convert' executable.
# It's important to set this before importing moviepy.
os.environ['IMAGEMAGICK_BINARY'] = '/usr/bin/convert' # Or /usr/bin/magick if convert doesn't work
```

```
# Import necessary libraries
from moviepy.editor import VideoFileClip, CompositeVideoClip, ImageClip
from moviepy.video.tools.drawing import color_gradient
from moviepy.editor import TextClip
from google.colab import files
from PIL import ImageFont, ImageDraw, Image
import numpy as np # Import numpy
import warnings # Import the warnings module

# Suppress UserWarnings from moviepy (related to ffmpeg_reader)
warnings.filterwarnings('ignore', category=UserWarning, module='moviepy.video.io.ffmpeg_reader')

# Mandatory output
print("Name: Prajwal Chaulagain\n")
print("Symbol No: 80010773\n")
print("Please upload a short video file (.mp4):")
# Upload the video file
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
# 1. Signal Representation: Get video properties
video = VideoFileClip(file_name)
frame_rate = video.fps
frame_count = int(frame_rate * video.duration)
# Display the frame rate and total frame count
print(f"Frame Rate: {frame_rate} fps")
print(f"Total Frames: {frame_count}")
# 2. Trim unwanted sections (Trim first 10 seconds or the video's full duration)
start_time = 0 # Starting from 0 seconds

# Calculate end_time. If it's the full duration, slightly reduce it to avoid warnings.
calculated_end_time = min(10, video.duration)
# Subtract a small epsilon (e.g., 0.05 seconds) if we are trimming to the end
if calculated_end_time == video.duration:
    # Ensure end_time doesn't go below start_time if video is very short
    end_time = max(0, video.duration - 0.05) # Increased epsilon for robustness
```

```
else:
    end_time = calculated_end_time
trimmed = video.subclip(start_time, end_time)


# 3. Use Pillow to overlay custom text (Position and font size can be customized)
def make_text_image(video_width, video_height):
  txt = "Student Edit"
# Create an image frame using PIL with video's width and height
  img = Image.new("RGBA", (video_width, video_height), (0, 0, 0, 0))
  draw = ImageDraw.Draw(img)
  font = ImageFont.load_default() # Use default font for PIL
  # Calculate text size for centering using textbbox
  left, top, right, bottom = draw.textbbox((0, 0), txt, font=font)
  text_w = right - left
  text_h = bottom - top
  text_position = ((img.width - text_w) // 2, (img.height - text_h) // 2)
  draw.text(text_position, txt, font=font, fill="white")
  return img
# 4. Apply text as a frame
# First, get the trimmed video dimensions for the text image
video_width, video_height = trimmed.size
# Create the static text image using Pillow
text_image_pil = make_text_image(video_width, video_height)
# Convert the PIL image to a MoviePy ImageClip, set duration and position
txt_clip = ImageClip(np.array(text_image_pil)).set_duration(trimmed.duration).set_pos("center") # Explicitly
# convert to NumPy array
# 5. Composite the text with the trimmed video
final_video = CompositeVideoClip([trimmed, txt_clip])
# 6. Save the edited video
final_video.write_videofile("edited_output.mp4", codec="libx264", audio_codec="aac")
print("Video processing complete. Output saved as 'edited_output.mp4'.")
```

Output:

```
Name: Prajwal Chaulagain

Symbol No: 80010773

Please upload a short video file (.mp4):
Choose Files Video.mp4
Video.mp4(video/mp4) - 1376689 bytes, last modified: 3/23/2026 - 100% done
Saving Video.mp4 to Video (5).mp4
Frame Rate: 30.0 fps
Total Frames: 264
Moviepy - Building video edited_output.mp4.
MoviePy - Writing audio in edited_outputTEMP_MPY_wvf_snd.mp4
MoviePy - Done.
Moviepy - Writing video edited_output.mp4

Moviepy - Done !
Moviepy - video ready edited_output.mp4
Video processing complete. Output saved as 'edited_output.mp4'.
```



Task 9: Simulate Mathematical Animation.

Objective: To program the motion of a graphic using trigonometric functions to simulate circular motion.

Tasks: Write a Python program to simulate mathematical animation. The program should:

• Generate a graphic object.

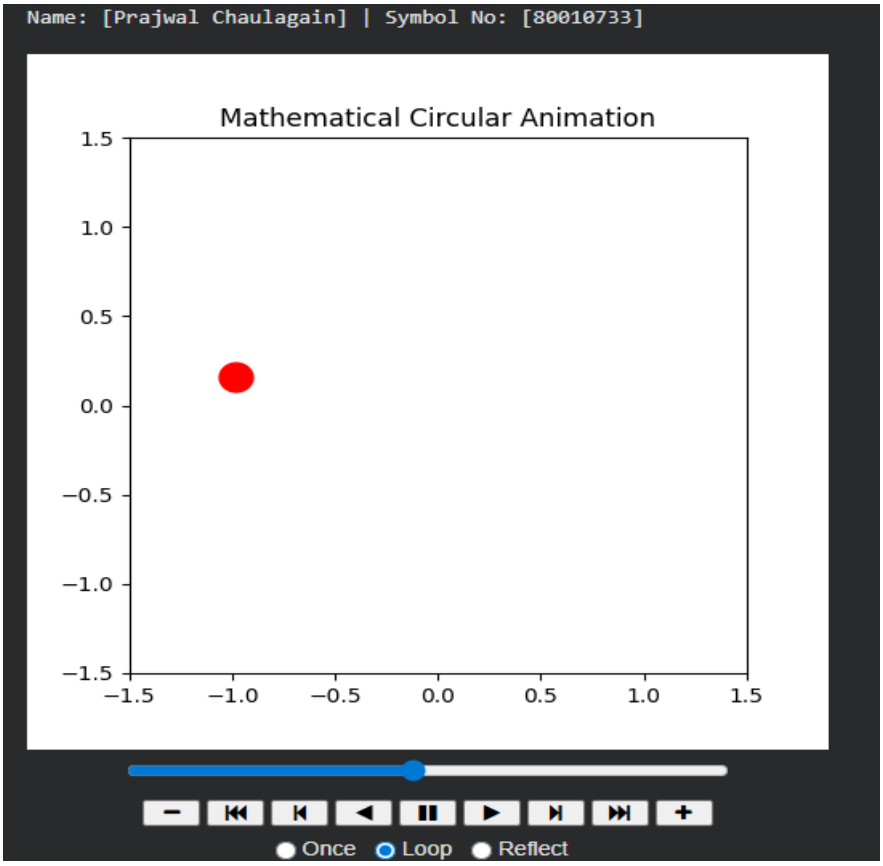
• Calculate its positional coordinates over time using Sine and Cosine functions.

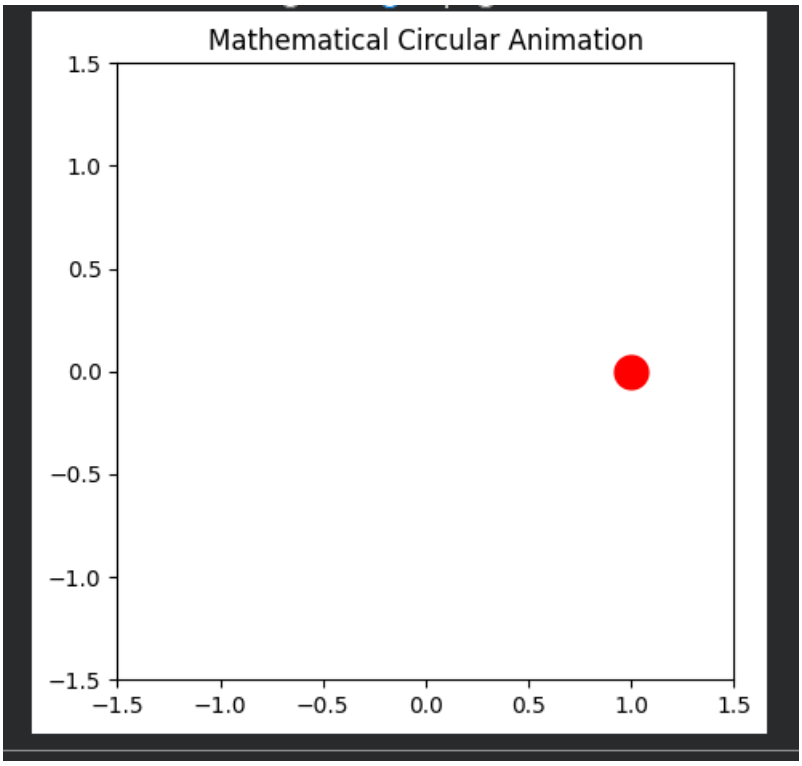
• Render and display the object moving in a continuous circular animation.

Code:

```
# --- Package Installation ---
!pip install matplotlib numpy
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010733]\n")
fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.5, 1.5)
ax.set_aspect('equal')
ax.set_title("Mathematical Circular Animation")
# Graphic object (a point)
point, = ax.plot([], [], 'ro', markersize=15)
def init():
  point.set_data([], [])
  return point,
def update(frame):
# Calculate positional coordinates using Sine and Cosine
  x = np.cos(frame)
  y = np.sin(frame)
  point.set_data([x], [y])
  return point,
# Render animation (0 to 2*Pi)
frames = np.linspace(0, 2 * np.pi, 60)
ani = FuncAnimation(fig, update, frames=frames, init_func=init, blit=True, interval=50)
# Display in Colab
HTML(ani.to_jshtml())
```

Output:


Name: [Prajwal Chaulagain] | Symbol No: [80010733]
Mathematical Circular Animation
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
Once
Loop
Reflect


Mathematical Circular Animation
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5

Task 10: Simulate Timeline & Tweening Animation.

Objective: To implement linear in-betweening (tweening) techniques for timeline-based animation.

Tasks: Write a Python program to simulate tweening animation. The program should:

• Define start and end states (coordinates/properties) for an object.

• Apply linear tweening to generate the intermediate frames automatically.

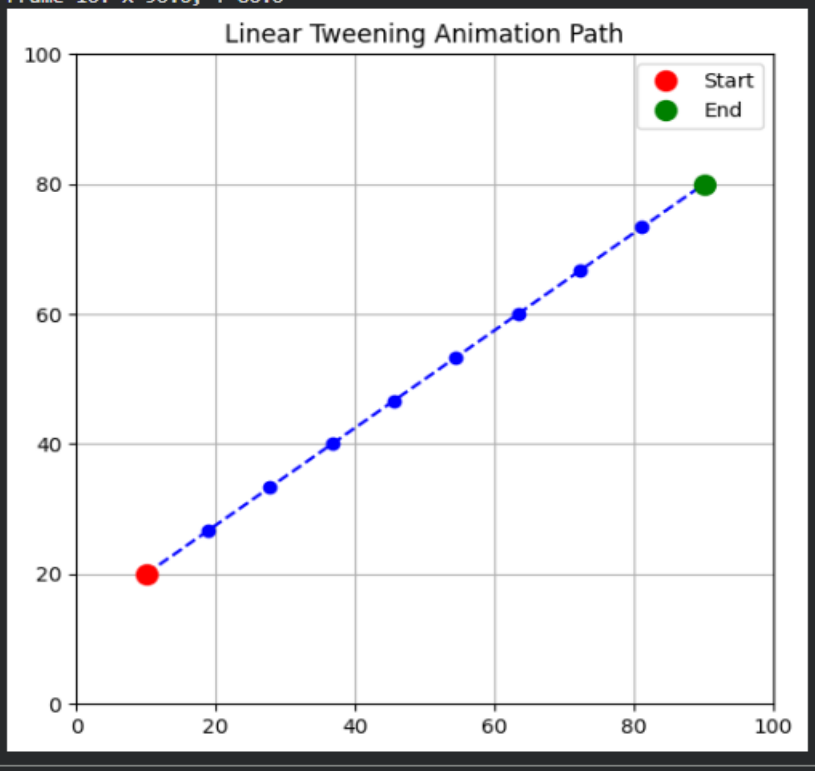
• Display the resulting smooth animation sequence.

Code:

```
# --- Package Installation ---
!pip install matplotlib numpy
import numpy as np
import matplotlib.pyplot as plt
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
# Define Start and End States
start_coord = np.array([10, 20])
end_coord = np.array([90, 80])
total_frames = 10
# Apply linear tweening to generate intermediate frames automatically
tween_frames = np.linspace(start_coord, end_coord, total_frames)
print("Generated Tweening Coordinates:")
for i, frame in enumerate(tween_frames):
  print(f"Frame {i+1}: X={frame[0]:.1f}, Y={frame[1]:.1f}")
# Display the resulting path
plt.figure(figsize=(6, 6))
plt.plot(tween_frames[:, 0], tween_frames[:, 1], marker='o', linestyle='--', color='blue')
plt.plot(start_coord[0], start_coord[1], 'ro', markersize=10, label="Start")
plt.plot(end_coord[0], end_coord[1], 'go', markersize=10, label="End")
plt.title("Linear Tweening Animation Path")
plt.xlim(0, 100); plt.ylim(0, 100)
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Name: [Prajwal Chaulagain] | Symbol No: [80010773]

Generated Tweening Coordinates:
Frame 1: X=10.0, Y=20.0
Frame 2: X=18.9, Y=26.7
Frame 3: X=27.8, Y=33.3
Frame 4: X=36.7, Y=40.0
Frame 5: X=45.6, Y=46.7
Frame 6: X=54.4, Y=53.3
Frame 7: X=63.3, Y=60.0
Frame 8: X=72.2, Y=66.7
Frame 9: X=81.1, Y=73.3
Frame 10: X=90.0, Y=80.0
```



Task 11: Simulate GIF Creation.

Objective: To export frame-by-frame sequences into a compiled animated GIF.

Tasks: Write a Python program to create an animated GIF. The program should:

• Generate or load a sequence of frame-by-frame images.

• Compile the individual sequences into a single animation timeline.

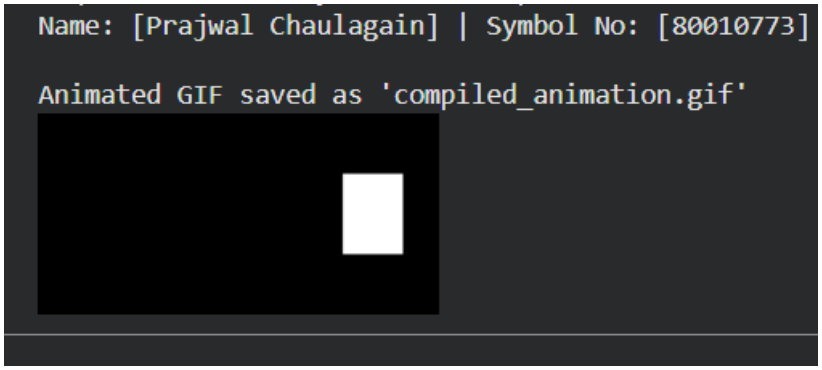
• Export and save the compiled sequence as an animated .gif file.

Code:

```
# --- Package Installation ---
!pip install imageio numpy
import imageio
import numpy as np
```

```
from IPython.display import Image
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
frames = []
# Generate a sequence of frames (moving a square block)
for i in range(20):
  frame = np.zeros((100, 200), dtype=np.uint8)
  # Move the white block to the right
  start_col = i * 8
  frame[30:70, start_col:start_col+30] = 255
  frames.append(frame)
# Compile and export as an animated GIF
gif_path = "compiled_animation.gif"
imageio.mimsave(gif_path, frames, fps=10)
print(f"Animated GIF saved as '{gif_path}'")
# Display the GIF inside Colab
Image(open(gif_path, 'rb').read())
```

Output:



Task 12: Simulate Entropy Coding (RLE).

Objective: To isolate and implement the Run-Length Encoding (RLE) algorithm for lossless data compression.

Tasks: Write a Python program to simulate RLE. The program should:

• Accept input data (text string or binary sequence).

• Apply the Run-Length Encoding algorithm to compress the data.

• Display the original size, the compressed output, and the compression ratio.

Code:

```
# --- Package Installation ---
# No external packages required. Uses standard Python.
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
def rle_compress(data):
  if not data:
    return ""
  encoded = []
  count = 1
  # Run-Length Encoding logic
  for i in range(1, len(data)):
    if data[i] == data[i - 1]:
      count += 1
    else:
      encoded.append(f"{data[i - 1]}{count}")
      count = 1
  # Append the last run of characters after the loop finishes
  encoded.append(f"{data[-1]}{count}")
  return "".join(encoded)
# Test Data
input_data =
"WWWWWWWWWWWWBWWWWWWWWWWWWBBBWWWWWWWWWWWWWWWWWW
WWWWWWWWBWWWWWWWWWWWWW"
compressed_data = rle_compress(input_data)
original_size = len(input_data)
compressed_size = len(compressed_data)
print(f"Original Data: {input_data}")
print(f"Compressed Output: {compressed_data}\n")
print(f"Original Size: {original_size} characters")
print(f"Compressed Size: {compressed_size} characters")
print(f"Compression Ratio: {original_size / compressed_size:.2f}")
```

Output:

```
Name: [Prajwal Chaulagain] | Symbol No: [80010773]

Original Data: WWWWWWWWWWWBWWWWWWWWWWWBBBWWWWWWWWWWWWWWWWWWWWWBWWWWWWWWWWWW
Compressed Output: W12B1W12B3W24B1W14

Original Size: 67 characters
Compressed Size: 18 characters
Compression Ratio: 3.72
```

Task 13: Simulate Entropy Coding (Huffman).

Objective: To isolate and implement the Huffman encoding algorithm, including building the binary tree.

Tasks: Write a Python program to simulate Huffman Encoding. The program should:

• Calculate character frequencies from an input string.

• Build a Huffman binary tree based on those frequencies.

• Generate and display the Huffman encoded binary string.

Code:

```
# --- Package Installation ---
# No external packages required. Uses standard Python collections.
import heapq
from collections import Counter
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
class Node:
  def __init__(self, char, freq):
    self.char = char
    self.freq = freq
    self.left = None
    self.right = None
  def __lt__(self, other):
    return self.freq < other.freq
def build_huffman_tree(text):
    freq = Counter(text)
    heap = [Node(char, f) for char, f in freq.items()]
```

```
    heapq.heapify(heap)
    while len(heap) > 1:
      left = heapq.heappop(heap)
      right = heapq.heappop(heap)
      merged = Node(None, left.freq + right.freq)
      merged.left = left
      merged.right = right
      heapq.heappush(heap, merged)
    return heap[0]
def generate_huffman_codes(node, current_code, codes):
  if node:
    if node.char is not None:  # It's a leaf node, assign the code
      codes[node.char] = current_code
    else:  # It's an internal node, continue traversing children
      generate_huffman_codes(node.left, current_code + "0", codes)
      generate_huffman_codes(node.right, current_code + "1", codes)

# Input data
input_string = "multimedia computing laboratory"
root = build_huffman_tree(input_string)
codes = {}
generate_huffman_codes(root, "", codes)
encoded_string = "".join(codes[char] for char in input_string)

print(f"Original String: '{input_string}'")
print("\nHuffman Codes Dictionary:")
for char, code in codes.items():
  print(f" '{char}': {code}")
print(f"\nEncoded Binary String:\n{encoded_string}")
```

Output:

```
Name: [Prajwal Chaulagain] | Symbol No: [80010773]

Original String: 'multimedia computing laboratory'

Huffman Codes Dictionary:
 't': 000
 'a': 001
 'm': 010
 'l': 0110
 'g': 01110
 'n': 01111
 ' ': 1000
 'r': 1001
 'u': 1010
 'e': 10110
 'p': 10111
 'b': 11000
 'd': 11001
 'c': 11010
 'y': 11011
 'o': 1110
 'i': 1111

Encoded Binary String:
010101001100001111010101101100111110011000110101110010101111010000111101111011101000011000111000111010011
```

Task 14: Simulate Lossy Compression (JPEG).

Objective: To implement JPEG compression and decompression using OpenCV.

Tasks: Write a Python program to simulate JPEG-style lossy compression. The program should:

• Accept a digital image as input.

• Implement JPEG compression using OpenCV parameters to simulate lossy encoding.

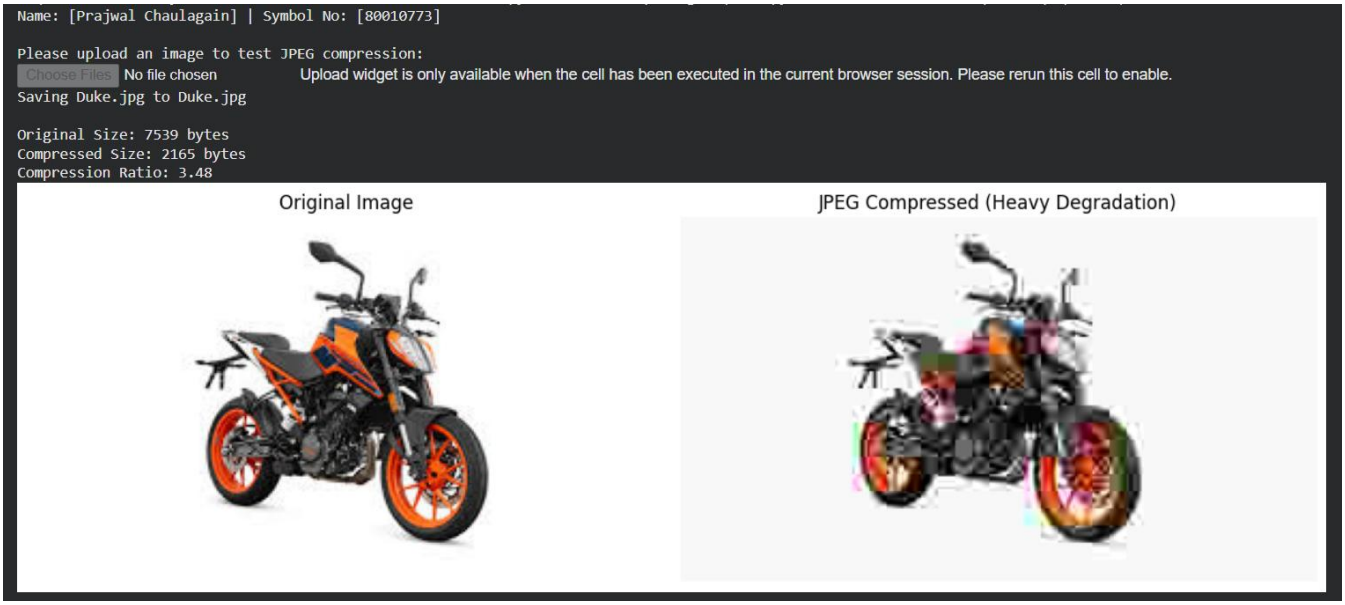
• Decompress and display the image to observe the visual degradation and file size differences.

Code:

```
# --- Package Installation ---
!pip install opencv-python matplotlib numpy
import cv2
import matplotlib.pyplot as plt
import os
from google.colab import files
# Mandatory output
print("Name: [Prajwal Chaulagain] | Symbol No: [80010773]\n")
```

```
print("Please upload an image to test JPEG compression:")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
# Load original image
image = cv2.imread(file_name)
original_size = os.path.getsize(file_name)
# Implement JPEG compression using OpenCV parameters (Quality: 5 out of 100)
encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 5]
result, encimg = cv2.imencode('.jpg', image, encode_param)
# Decompress back to image
decimg = cv2.imdecode(encimg, 1)
compressed_size = encimg.nbytes
# Display results
print(f"\nOriginal Size: {original_size} bytes")
print(f"Compressed Size: {compressed_size} bytes")
print(f"Compression Ratio: {original_size / compressed_size:.2f}")

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(decimg, cv2.COLOR_BGR2RGB))
plt.title("JPEG Compressed (Heavy Degradation)")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:



Task 15: Web-Based Multimedia Integration.

Objective: To design and implement a web-based multimedia interface, integrating various media types into a single application.

Tasks: Write HTML, CSS, and JavaScript code to develop a web multimedia page. The program should:

• Insert text, images, tables, dropdown lists, radio buttons, and a submit button.

• Embed audio and video elements directly into the page.

• Implement a drag-and-drop event and a form action to simulate sending an email.

Code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Web-Based Multimedia Integration</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        margin: 20px;
      }
```

```
    .student-info {
      background-color: #f4f4f4;
      padding: 10px;
      border-left: 5px solid #007bff;
      margin-bottom: 20px;
    }
    table,
    th,
    td {
      border: 1px solid black;
      border-collapse: collapse;
      padding: 10px;
    }
    .dropzone {
      width: 300px;
      height: 100px;
      border: 2px dashed #333;
      margin-top: 15px;
      padding: 10px;
      text-align: center;
    }
    .form-section {
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div class="student-info">
    <strong>Name:</strong> Prajwal Chaulagain <br />
    <strong>Symbol No:</strong> 80010773
  </div>
  <h2>1. Text, Images, and Tables</h2>
  <p>Embedding standard digital media via HTML.</p>
  <img src="Duke.jpg" alt="Student Image" width="200" /><br /><br />
  <table>
```

```
  <tr>
    <th>Compression Type</th>
    <th>Algorithm</th>
  </tr>
  <tr>
    <td>Lossless</td>
    <td>Huffman / RLE</td>
  </tr>
  <tr>
    <td>Lossy</td>
    <td>JPEG (DCT-based)</td>
  </tr>
</table>
<h2>2. Dropdown List and Radio Buttons</h2>
<div class="form-section">
  <label for="compression">Select Compression Type:</label>
  <select id="compression">
    <option value="lossless">Lossless</option>
    <option value="lossy">Lossy</option>
  </select>
  <p>Choose algorithm:</p>
  <input type="radio" id="huffman" name="algorithm" value="Huffman" />
  <label for="huffman">Huffman</label><br />
  <input type="radio" id="rle" name="algorithm" value="RLE" />
  <label for="rle">RLE</label><br />
  <input type="radio" id="jpeg" name="algorithm" value="JPEG" />
  <label for="jpeg">JPEG</label><br />
</div>
<h2>3. Audio and Video</h2>
<audio controls>
  <source src="sample.mp3" type="audio/mpeg" />
  Your browser does not support the audio element.
</audio>
<br /><br />
<video width="320" height="240" controls>
```

```
  <source src="Video.mp4" type="video/mp4" />
  Your browser does not support the video tag.
</video>
<h2>4. Drag-and-Drop Simulation</h2>
<div id="dropzone" class="dropzone">Drag and drop a file here</div>
<p id="drop-result"></p>
<h2>5. Form Submission Simulation</h2>
<div class="form-section">
  <form id="emailForm">
    <label for="toEmail">To:</label>
    <input type="email" id="toEmail" required /><br /><br />
    <label for="subject">Subject:</label>
    <input type="text" id="subject" required /><br /><br />
    <textarea
      id="message"
      rows="4"
      cols="50"
      placeholder="Write your message
          here..."
      required
    ></textarea
    ><br /><br />
    <button type="submit">Send Email</button>
  </form>
  <p id="form-result"></p>
</div>
<script>
  // Drag-and-Drop logic
  const dropzone = document.getElementById("dropzone");
  const dropResult = document.getElementById("drop-result");

  dropzone.addEventListener("dragover", (e) => {
    e.preventDefault();
    dropzone.style.borderColor = "green";
  });
```

```
    dropzone.addEventListener("dragleave", () => {
      dropzone.style.borderColor = "#333";
    });
    dropzone.addEventListener("drop", (e) => {
      e.preventDefault();
      const files = e.dataTransfer.files;
      dropResult.textContent = `File "${files[0].name}" dropped successfully!`;
      dropzone.style.borderColor = "#333";
    });
    // Form submission simulation
    const emailForm = document.getElementById("emailForm");
    const formResult = document.getElementById("form-result");
    emailForm.addEventListener("submit", (e) => {
      e.preventDefault();
      const to = document.getElementById("toEmail").value;
      const subject = document.getElementById("subject").value;
      const message = document.getElementById("message").value;
      formResult.textContent = `Email sent to ${to} with subject "${subject}" and message:
(simulation)`;
      emailForm.reset();
    });
  </script>
 </body>
</html>
```

Output:

Name: Prajwal Chaulagain
Symbol No: 80010773

1. Text, Images, and Tables

Embedding standard digital media via HTML.

Compression Type	Algorithm
Lossless	Huffman / RLE
Lossy	JPEG (DCT-based)

2. Dropdown List and Radio Buttons

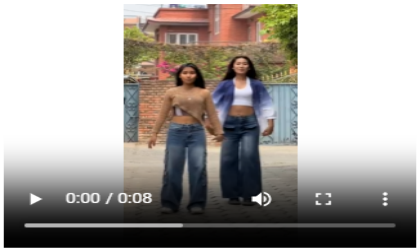
Select Compression Type: Lossless

Choose algorithm:

- (•) Huffman
- ( ) RLE
- ( ) JPEG

3. Audio and Video

▶ 0:00 / 2:16



4. Drag-and-Drop Simulation

Drag and drop a file here

File "Flower.jpg" dropped successfully!

5. Form Submission Simulation

To: prajwalchaulagain6@gmail.c

Subject: Multimedia Computing

This is a Lab Report for Multimedia Computing.

Send Email

Email sent to prajwalchaulagain6@gmail.com with subject "Multimedia Computing" and message: (simulation)